

苏州大学实验报告

院系	计算机学院	年级专业	21 计科	姓名	方浩楠	学号	2127405048
课程名称	操作系统课程实践					成绩	
指导教师	王红玲	同组实验者	无	实验日期	2024.4.10		

实 验 名 称

实验 6 内核模块实验

一. 实验目的

- (1) 掌握利用内核模块机制实现较复杂功能的办法。
- (2) 学习并掌握/proc 文件系统的创建。

二. 实验内容

利用内核模块在/proc 模块下创建 proc_example 目录，并在该目录下创建三个普通文件(foo, bar, jiffies)和一个链接文件(jiffies_too)。

三. 实验步骤和结果

1. 使用内核模块创建一个文件节点:

源代码如下:

```
procfs_example.c:
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/proc_fs.h>
#include <linux/jiffies.h>
#include <linux/sched.h>
#include <linux/uaccess.h>
#include <linux/seq_file.h>
#include <linux/fs.h>

#define MODULE_VERS "1.0"
#define MODULE_NAME "procfs_example"
#define FOOBAR_LEN 8

struct fb_data_t {
    char name[FOOBAR_LEN + 1];
    char value[FOOBAR_LEN + 1];
};
```

```

static struct proc_dir_entry *example_dir, *foo_file, *bar_file, *jiffies_file, *symlink;
struct fb_data_t foo_data, bar_data;
int foo_len, foo_temp, bar_len, bar_temp;
int jiff_temp = -1;
char tempstr[FOOBAR_LEN * 2 + 5];

// jiffies 文件操作函数
ssize_t read_jiffies_proc(struct file *filp, char __user *buf, size_t count, loff_t *offp) {
    char tempstring[100] = "";
    int bytes;
    if (jiff_temp != 0)
        jiff_temp = sprintf(tempstring, "jiffies=%ld\n", jiffies);
    if (count > jiff_temp)
        count = jiff_temp;
    jiff_temp = jiff_temp - count;
    bytes = copy_to_user(buf, tempstring, count) ? -EFAULT : count;
    if (bytes == 0)
        jiff_temp = -1;
    return bytes;
}

// proc_ops 结构定义
static const struct proc_ops jiffies_proc_ops = {
    .proc_read = read_jiffies_proc,
};

// foo 文件操作函数
ssize_t read_foo_proc(struct file *filp, char __user *buf, size_t count, loff_t *offp) {
    if (count > foo_temp)
        count = foo_temp;
    foo_temp = foo_temp - count;
    strcpy(tempstr, foo_data.name);
    strcat(tempstr, "=");
    strcat(tempstr, foo_data.value);
    strcat(tempstr, "\n");
    if (copy_to_user(buf, tempstr, count))
        return -EFAULT;
    if (count == 0)
        foo_temp = foo_len + 4;
    return count;
}

ssize_t write_foo_proc(struct file *filp, const char __user *buf, size_t count, loff_t *offp) {
    int len = count > FOOBAR_LEN ? FOOBAR_LEN : count;
    if (copy_from_user(foo_data.value, buf, len))
        return -EFAULT;
}

```

```

    foo_data.value[len-1] = '\0';
    foo_len = strlen(foo_data.name) + strlen(foo_data.value);
    foo_temp = foo_len + 4;
    return len;
}

static const struct proc_ops foo_proc_ops = {
    .proc_read = read_foo_proc,
    .proc_write = write_foo_proc,
};

// bar 文件操作函数
ssize_t read_bar_proc(struct file *filp, char __user *buf, size_t count, loff_t *offp) {
    if (count > bar_temp)
        count = bar_temp;
    bar_temp = bar_temp - count;
    strcpy(tempstr, bar_data.name);
    strcat(tempstr, "=");
    strcat(tempstr, bar_data.value);
    strcat(tempstr, "\n");
    if (copy_to_user(buf, tempstr, count))
        return -EFAULT;
    if (count == 0)
        bar_temp = bar_len + 4;
    return count;
}

ssize_t write_bar_proc(struct file *filp, const char __user *buf, size_t count, loff_t *offp) {
    int len = count > FOOBAR_LEN ? FOOBAR_LEN : count;
    if (copy_from_user(bar_data.value, buf, len))
        return -EFAULT;
    bar_data.value[len-1] = '\0';
    bar_len = strlen(bar_data.name) + strlen(bar_data.value);
    bar_temp = bar_len + 4;
    return len;
}

static const struct proc_ops bar_proc_ops = {
    .proc_read = read_bar_proc,
    .proc_write = write_bar_proc,
};

// 模块 init 函数
static int __init init_procfs_example(void) {
    example_dir = proc_mkdir(MODULE_NAME, NULL);
    if (!example_dir)

```

```

        return -ENOMEM;

jiffies_file = proc_create("jiffies", 0444, example_dir, &jiffies_proc_ops);
if (!jiffies_file)
    goto out_jiffies;

strcpy(foo_data.name, "foo");
strcpy(foo_data.value, "foo");
foo_len = strlen(foo_data.name) + strlen(foo_data.value);
foo_temp = foo_len + 4;
foo_file = proc_create("foo", 0444, example_dir, &foo_proc_ops);
if (!foo_file)
    goto out_foo;

strcpy(bar_data.name, "bar");
strcpy(bar_data.value, "bar");
bar_len = strlen(bar_data.name) + strlen(bar_data.value);
bar_temp = bar_len + 4;
bar_file = proc_create("bar", 0444, example_dir, &bar_proc_ops);
if (!bar_file)
    goto out_bar;

symlink = proc_symlink("jiffies_too", example_dir, "jiffies");
if (!symlink)
    goto out_symlink;

printk(KERN_INFO "%s%s initialised\n", MODULE_NAME, MODULE_VERS);
return 0;

out_symlink:
    remove_proc_entry("bar", example_dir);
out_bar:
    remove_proc_entry("foo", example_dir);
out_foo:
    remove_proc_entry("jiffies", example_dir);
out_jiffies:
    remove_proc_entry(MODULE_NAME, NULL);
    return -ENOMEM;
}

// 模块 cleanup 函数
static void __exit cleanup_procfs_example(void) {
    remove_proc_entry("jiffies_too", example_dir);
    remove_proc_entry("bar", example_dir);
    remove_proc_entry("foo", example_dir);
    remove_proc_entry("jiffies", example_dir);
}

```

```

remove_proc_entry(MODULE_NAME, NULL);
printk(KERN_INFO "%s%s removed\n", MODULE_NAME, MODULE_VERS);
}

```

```

MODULE_LICENSE("GPL");
module_init(init_procfs_example);
module_exit(cleanup_procfs_example);
MODULE_DESCRIPTION("proc filesystem example");

```

Makefile:

```

CONFIG_MODULE_SIG=n
obj-m := procfs_example.o
KDIR := /usr/src/linux-headers-$(shell uname -r )
PWD := $(shell pwd)
all:
    make -C $(KDIR) M=$(PWD) modules
clean:
    make -C $(KDIR) M=$(PWD) clean

```

编译代码:

```

fhn@ubuntu:~/OS_experiment/procfs$ make
make -C /usr/src/linux-headers-5.15.0-101-generic M=/home/fhn/OS_experiment/procfs modules
make[1]: 进入目录 "/usr/src/linux-headers-5.15.0-101-generic"
CC [M] /home/fhn/OS_experiment/procfs/procfs_example.o
MODPOST /home/fhn/OS_experiment/procfs/Module.symvers
CC [M] /home/fhn/OS_experiment/procfs/procfs_example.mod.o
LD [M] /home/fhn/OS_experiment/procfs/procfs_example.ko
BTF [M] /home/fhn/OS_experiment/procfs/procfs_example.ko
Skipping BTF generation for /home/fhn/OS_experiment/procfs/procfs_example.ko due to unavailability of vmlinux
make[1]: 离开目录 "/usr/src/linux-headers-5.15.0-101-generic"

```

装入内核模块:

```

fhn@ubuntu:~/OS_experiment/procfs$ sudo insmod procfs_example.ko
[sudo] fhn 的密码:
fhn@ubuntu:~/OS_experiment/procfs$

```

```

fhn@ubuntu:~/OS_experiment/procfs$ sudo insmod procfs_example.ko
[sudo] fhn 的密码:
fhn@ubuntu:~/OS_experiment/procfs$ sudo lsmod | grep procfs
procfs_example      20480  0
fhn@ubuntu:~/OS_experiment/procfs$

```

查看 procfs_example 目录及其中的各个文件的内容:

```

fhn@ubuntu:~/OS_experiment$ cd /proc/procfs_example/
fhn@ubuntu:/proc/procfs_example$ cat bar foo jiffies jiffies_too
bar='bar'
foo='foo'
jiffies=4295463432
jiffies=4295463432
fhn@ubuntu:/proc/procfs_example$

```


此“读写”这个概念不适用于它。通过这个链接读取数据实际上是读取 jiffies 文件的内容，所以它也是只读的。

bar 和 foo 文件：这两个文件的属性取决于它们被创建时指定的权限和操作。在你的示例代码中，它们最初被创建为只读的,这是因为代码中

```
foo_file = proc_create("foo", 0444, example_dir, &foo_proc_ops);
```

```
bar_file = proc_create("bar", 0444, example_dir, &bar_proc_ops);
```

这两行保证了这两个文件只读

由于文件权限被设置为只读（0444），那么无论写操作函数如何定义，文件都只能被读取。